

FUNDAMENTOS DE VISÃO COMPUTACIONAL

CIÊNCIA DA COMPUTAÇÃO

SEGMENTAÇÃO DE IMAGENS

COMPONENTES CONECTADOS (BLOBS)

Segmentação Watershed

CONTANDO OBJETOS EM UMA IMAGEM

```
import cv2 as cv
import numpy as np

imgRice = cv.imread("rice.png", 0)

imgBinary = cv.adaptiveThreshold (imgRice, 255.0,
                                cv.ADAPTIVE_THRESH_MEAN_C, cv.THRESH_BINARY, 51, -20.0)

cv.imshow("Imagem Binarizada", imgBinary)

# Aplicação de erosão
kernel = np.ones((5,5),np.uint8)
imgErosion = cv.erode(imgBinary, kernel)
cv.imshow("Imagem com Erosão", imgErosion)

contours, _ = cv.findContours(imgErosion, cv.RETR_EXTERNAL, cv.CHAIN_APPROX_SIMPLE)
imgContour = cv.cvtColor(imgRice, cv.COLOR_GRAY2BGR)

cv.drawContours(imgContour, contours, -1, (0, 0, 255), 2)

print("Total de objetos detectados: ", len(contours))
cv.imshow("Objetos detectados", imgContour)

cv.waitKey(0)
cv.destroyAllWindows()
```

Carrega imagem em tons de cinza

ADAPTIVE_THRESH_MEAN_C: método de thresholding (média menos a constante C)
THRESH_BINARY: tipo de threshold (acima do limiar = branco e abaixo = preto)
51: o tamanho da janela de vizinhança para calcular o limiar.
-20.0: a constante C

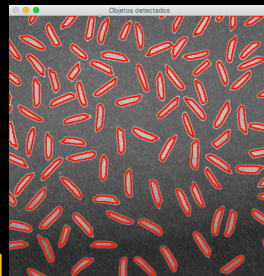
A matriz 5x5 (kernel) é usada para dilatar a imagem binarizada.

Erosão da imagem

Detecta os contornos

Imagem original (grayscale) é convertida para colorida para permitir a visualização dos contornos.

Os contornos são desenhados em vermelho (0,0,255) com uma espessura de 2 pixels



Total de objetos detectados: 101

cv.dilate
x
cv.erode

CONTANDO OBJETOS EM UMA IMAGEM

```
import cv2 as cv
import numpy as np

imgRice = cv.imread("coins.jpeg", 0)

_,imgBinary = cv.threshold(imgRice,0,255,cv.THRESH_BINARY_INV+cv.THRESH_OTSU)
cv.imshow("Imagem Binarizada", imgBinary)

## Dilatamento da imagem para reduzir ruído
kernel = np.ones((5,5),np.uint8)
imgDilated = cv.dilate(imgBinary, kernel)
cv.imshow("Imagem com Erosão", imgDilated)

contours, _ = cv.findContours(imgDilated, cv.RETR_EXTERNAL, cv.CHAIN_APPROX_SIMPLE)
imgContour = cv.cvtColor(imgRice, cv.COLOR_GRAY2BGR)

cv.drawContours(imgContour, contours, -1, (0, 0, 255), 2)

print("Total de objetos detectados: ", len(contours))
cv.imshow("Objetos detectados", imgContour)

cv.waitKey(0)
cv.destroyAllWindows()
```

Carrega imagem em tons de cinza

Otsu será usado para determinar automaticamente o melhor limiar e os pixels.

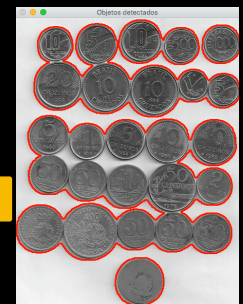
A matriz 5x5 (kernel) é usada para dilatar a imagem binarizada.

Dilatação da imagem

Detecta os contornos

Imagem original (grayscale) é convertida para colorida para permitir a visualização dos contornos.

Os contornos são desenhados em vermelho (0,0,255) com uma espessura de 2 pixels

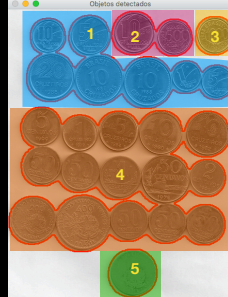
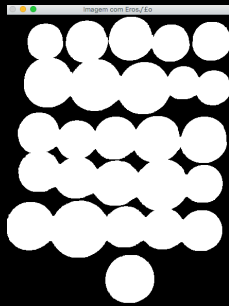


Total de objetos detectados: 5

cv.dilate
x
cv.erode

CONTANDO OBJETOS EM UMA IMAGEM

Problema...



Total de objetos detectados: 5

SEGMENTAÇÃO POR WATERSHED

COMPONENTES CONECTADOS

SEGMENTAÇÃO POR WATERSHED



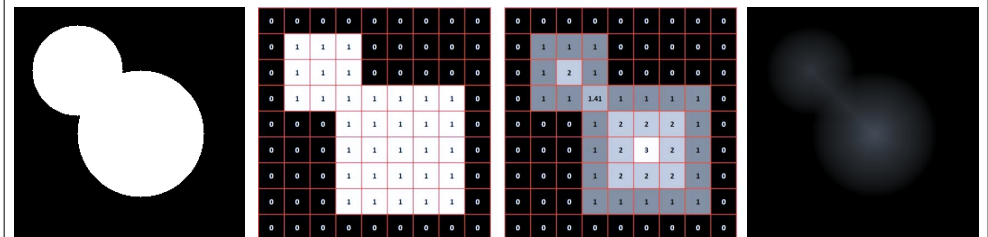
Initial image

Se inundarmos uma superfície partindo de seus mínimos e, se impedirmos a fusão das águas provenientes de diferentes fontes, particionamos a imagem em dois conjuntos distintos: as bacias hidrográficas e as linhas divisoras de águas (watershed lines).

Útil para segmentar objetos diferentes interconectados ou até sobrepostos.

Fonte: <https://people.cmm.minesparis.psl.eu/users/beucher/wtshed.html>

SEGMENTAÇÃO POR WATERSHED

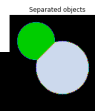
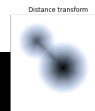
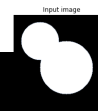


Binary image

Binary image matrix

Distance transform matrix

Distance transform



$$d(p_1, p_2) = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$$

DISTÂNCIA
EUCLIDIANA

Fonte: <https://datahacker.rs/007-opencv-projects-image-segmentation-with-watershed-algorithm/>

WATERSHED

```

1 import cv2
2 import numpy as np
3 from scipy import ndimage as ndi
4 from skimage.feature import peak_local_max
5 from skimage.segmentation import watershed
6 from skimage import color
7
8 img = cv2.imread("coins.jpeg")
9
10 filtro = cv2.pyrMeanShiftFiltering(img, 20, 40)
11 gray = cv2.cvtColor(filtro, cv2.COLOR_BGR2GRAY)
12
13 _, thresh = cv2.threshold(gray, 0, 255, cv2.THRESH_BINARY_INV | cv2.THRESH_OTSU)
14
15 # Preenchimento de falhas nos objetos binarizados
16 contornos, _ = cv2.findContours(thresh, cv2.RETR_TREE, cv2.CHAIN_APPROX_SIMPLE)
17 falhas = []
18 for con in contornos:
19     area = cv2.contourArea(con)
20     if area < 1000:
21         falhas.append(con)
22 cv2.drawContours(thresh, falhas, -1, 255, -1)
23 preencheda = thresh.copy() # copia para não perder dados e exibir thresh no final
24
25 # cálculo da distância da transformação
26 dist = cv2.distanceTransform(preencheda, distanceType=cv2.DIST_L2, maskSize=3, dstType=cv2.CV_8U)
27 dist = cv2.normalize(dist, None, alpha=0, beta=255, norm_type=cv2.NORM_MINMAX, dtype=cv2.CV_8U)
28 local_maxi = peak_local_max(dist, min_distance=20, labels=preencheda)
29 local_max = np.zeros_like(dist, dtype=bool)
30 local_max[tuple(local_maxi.T)] = True
31
32 # marcando os labels
33 markers = ndi.label(local_max, structure=np.ones((3,3)))[0]
34
35 #watershed
36 labels = watershed(-dist, markers, mask=thresh)
37 print("Objetos encontrados: ", len(np.unique(labels)) - 1)
38 imgWatershed = color.label2rgb(labels, bg_label=0)
39
40 cv2.imshow("1 Imagem original", img)
41 cv2.imshow("3 Imagem preencheda", thresh)
42 cv2.imshow("4 Calculo Dist.", dist)
43 cv2.imshow("5 Watershed", imgWatershed)
44
45 cv2.waitKey(0)
46 cv2.destroyAllWindows()

```

Pip3 install scikit-image

FILTRO QUE PRESERVA AS BORDAS DA IMAGEM, BASTANTE UTILIZADO COM WATERSHED

PREENCHE ÁREAS MENORES QUE 1000 PIXELS COM A COR BRANCA

DIST_L2 = DISTÂNCIA EUCLIDIANA SIMPLES

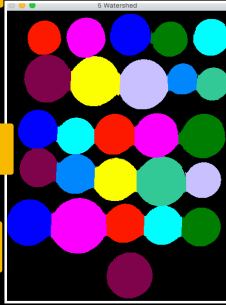
ENCONTRA OS MÁXIMOS LOCAIS (PICOS)

ATRIBUI CORES ALEATÓRIAS PARA CADA LABEL DA IMAGEM

VALOR NEGATIVO (-dist) PARA ORDENAR DE FORMA DECRESCENTE

QUANTIDADE DE OBJETOS DETECTADOS

Pip3 install scikit-image



Objetos encontrados: 26

CONTANDO OBJETOS EM UMA IMAGEM

